

EXPERIMENT – 1

PROBLEM STATEMENT: Create a menu-driven application that performs the following Natural Language Processing tasks using the pipeline() function of the Transformers library:

- 1.Sentiment Analysis
- 2.Text Generation
- 3.Summarization
- 4.Question Answering
- 5.Translation

OBJECTIVES

- 1.To understand the use of the Hugging Face Transformers pipeline() function.
- 2.To perform different NLP tasks using pre-trained Transformer models.
- 3.To implement a menu-driven NLP application.
- 4.To understand automatic model selection using pipeline().
- 5.To understand how specific models can be selected using the model parameter.
- 6.To compare the behavior of different models for the same NLP task.
- 7.To observe the behavior of a model when it is used for a task for which it was not designed.
- 8.To understand the use of max_new_tokens and temperature during text generation.

THEORY

1. Overview of the Experiment

The Transformers library provides pre-trained machine learning models for various Natural Language Processing tasks. The pipeline() function provides a simple interface for using these models without implementing the complete model architecture manually.

In this experiment, a menu-driven application is developed that allows the user to select different NLP operations.

The application performs:

Sentiment Analysis

Sentiment Analysis determines the sentiment expressed in a given text, such as **positive** or **negative**.

Example:

Input: I really enjoyed this movie.

Output: POSITIVE

Text Generation

Text Generation generates new text based on a given prompt. In this experiment, the GPT-2 model is used.

The max_new_tokens parameter controls the maximum number of new tokens generated, while temperature controls the randomness of the generated text.

Summarization

Summarization generates a shorter version of a longer input text while retaining important information.

Question Answering

Question Answering extracts an answer from a given context according to the question provided by the user.

Translation

Translation converts text from one language to another. In this experiment, English text is translated into Hindi.

2. Libraries Used**Transformers**

The transformers library is used to access pre-trained Transformer models and the pipeline() function.

```
from transformers import pipeline
```

PyTorch

PyTorch is required by the Transformers models for model execution.

```
torch
```

SentencePiece

SentencePiece is used by several Transformer models for tokenization.

```
sentencepiece
```

The required packages can be installed using:

```
!pip install transformers==4.53.0
```

```
!pip install torch
```

```
!pip install sentencepiece
```

3. Functions Used**pipeline()**

Creates an NLP pipeline for a particular task.

Example:

```
pipeline("sentiment-analysis")
```

input()

Takes input from the user.

```
text = input("Enter text: ")
```

print()

Displays the result on the screen.

classifier()

Used to perform sentiment analysis after creating a sentiment-analysis pipeline.

generator()

Used to generate text from a given prompt.

summarizer()

Used to summarize input text.

qa()

Used to answer a question from a given context.

translator()

Used to translate text from one language to another

WORKING OF PROGRAM

The program starts by importing the pipeline function from the Transformers library.

A separate function is created for each NLP task:

- 1 .sentiment_analysis()
- 2 .text_generation()
- 3 .summarization()
- 4 .question_answering()
- 5 .translation()

The user is presented with a menu.

1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Different Models
7. Model for Another Task
8. Exit

The selected option calls the corresponding function.

A. Pipeline Without Specifying Model

For example:

```
classifier = pipeline("sentiment-analysis")
```

Here, no model is explicitly specified. The pipeline automatically selects and downloads a suitable model.

The same approach is used for:

```
pipeline("summarization")
```

and:

```
pipeline("question-answering")
```

B. Using Specific Models

A model can also be explicitly specified using the model parameter.

For example:

```
translator = pipeline(  
    "translation",  
    model="Helsinki-NLP/opus-mt-en-hi"  
)
```

Different sentiment-analysis models can also be tested to compare their results.

C. Using a Model Intended for Another Task

The experiment also tests a model with a task for which it was not designed.

For example:

```
classifier = pipeline(  
    "sentiment-analysis",  
    model="gpt2"  
)
```

GPT-2 is intended for text generation. Using it directly for sentiment analysis demonstrates the importance of selecting a model appropriate for the required task.

D. Text Generation Parameters

The GPT-2 model is used for text generation:

```
generator = pipeline("text-generation", model="gpt2")
```

The following parameters are used:

```
result = generator(  
    text,  
    max_new_tokens=50,  
    temperature=0.7  
)
```

`max_new_tokens` specifies the maximum number of new tokens to generate.

`temperature` controls the randomness of the generated output.

Sentiment Analysis and Text Generation

```
===== MENU =====
1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit
```

[Fix Code](#)

Enter your choice: 1

[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f. Using a pipeline without specifying a model name and revision in production is not recommended.

Loading weights: 100%  104/104 [00:00<00:00, 4062.05it/s]

Enter text: I Love this Movie.

Result: [{'label': 'POSITIVE', 'score': 0.9998736381530762}]

```
===== MENU =====
1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit
```

Enter your choice: 2

Loading weights: 100%  148/148 [00:00<00:00, 4633.69it/s]

Enter prompt: Explain about Artificial Intelligence.

[transformers] Both `max\_new\_tokens` (=50) and `max\_length` (=50) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

Generated Text:

Explain about Artificial Intelligence.

We use three types of artificial intelligence:

intelligent (intelligent-but-not-intelligent) or general intelligence (intelligent-but-not-general)

We develop intelligent software that can be used to make

PROGRAM CODE

```
!pip install transformers==4.53.0
```

```
!pip install torch
```

```
!pip install sentencepiece
```

```
from transformers import pipeline
```

```
def sentiment_analysis():
```

```
    classifier = pipeline("sentiment-analysis")
```

```
    text = input("Enter text: ")
```

```
    result = classifier(text)
```

```
    print("\nResult:", result)
```

```
def text_generation():
```

```
    generator = pipeline("text-generation", model="gpt2")
```

```
    text = input("Enter prompt: ")
```

```
    result = generator(
```

```
        text,
```

```
        max_new_tokens=50,
```

```
        temperature=0.7
```

```
)  
print("\nGenerated Text:")  
print(result[0]["generated_text"])
```

```
def summarization():  
    summarizer = pipeline("summarization")  
    text = input("Enter long text:\n")  
    result = summarizer(text, max_length=50, min_length=20, do_sample=False)  
    print("\nSummary:")  
    print(result[0]["summary_text"])
```

```
def question_answering():  
    qa = pipeline("question-answering")  
    context = input("Enter Context:\n")  
    question = input("Enter Question: ")  
    result = qa(question=question, context=context)  
    print("\nAnswer:", result["answer"])
```

Translation And Exit...

===== MENU =====

1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit

Enter your choice: 5

```
/opt/anaconda3/lib/python3.13/site-packages/transformers/models/marian/tokenization_marian.py:175: UserWarning: Recommended: pip install sacremoses.
warnings.warn("Recommended: pip install sacremoses.")
```

```
Device set to use mps:0
```

Enter English text: Summarization is an NLP task used to generate a shorter and more meaningful version of a longer text while preserving its important information. It helps reduce the length of large documents, articles, reports, or paragraphs so that the main ideas can be understood quickly. In the Transformers library, the pipeline("summarization") function can be used to perform automatic text summarization using a pre-trained Transformer model. The input text is provided to the summarization pipeline, which processes the text and generates a concise summary. Parameters such as max_length and min_length can be used to control the length of the generated summary, while do_sample=False can be used to generate a deterministic summary.

Hindi Translation:

संक्षेपण एक NLP कार्य है जो लंबे पाठ के एक छोटे से और अधिक अर्थपूर्ण संस्करण तैयार करने के लिए प्रयोग किया जाता है। यह बड़े दस्तावेजों, आलेखों, रिपोर्टों, या पैराग्राफ की लंबाई को कम करने में मदद करता है ताकि मुख्य विचारों को जल्दी समझ में लिया जा सके। रूपांतरण लाइब्रेरी में, pipeline("summarization") फ़ंक्शन एक स्वचालित मॉडल उपयोग में लिया जा सकता है। आप पाठ का उपयोग करने के लिए पाठ का उपयोग किया जा सकता है, और संक्षेपण पेश किया जा सकता है, और संक्षेपण पेश किया जा सकता है। उदाहरण के लिए, उदाहरण के रूप में प्रयोग किया जा सकता है, उदाहरण के लिए।

===== MENU =====

1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit

Enter your choice: 6

Exiting...

```
def translation():
    translator = pipeline(
        "translation",
        model="Helsinki-NLP/opus-mt-en-hi"
    )
    text = input("Enter English text: ")
    result = translator(text)
    print("\nHindi Translation:")
    print(result[0]["translation_text"])

def different_models():
    text = input("Enter text: ")

    print("\nModel 1:")
    classifier = pipeline(
        "sentiment-analysis",
        model="distilbert-base-uncased-finetuned-sst-2-english"
    )
    result = classifier(text)
    print(result)
```

Summarization and Question Answering

```
===== MENU =====
1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit
Enter your choice: 3
No model was supplied, defaulted to sshleifer/distilbart-cnn-12-6 and revision a4f8f3e (https://huggingface.co/sshleifer/distilbart-cnn-12-6).
Using a pipeline without specifying a model name and revision in production is not recommended.
Device set to use mps:0
Enter long text:
Summarization is an NLP task used to generate a shorter and more meaningful version of a longer text while preserving its important information. It helps reduce the length of large documents, articles, reports, or paragraphs so that the main ideas can be understood quickly. In the Transformers library, the pipeline("summarization") function can be used to perform automatic text summarization using a pre-trained Transformer model. The input text is provided to the summarization pipeline, which processes the text and generates a concise summary. Parameters such as max_length and min_length can be used to control the length of the generated summary, while do_sample=False can be used to generate a deterministic summary.

Summary:
In Transformers library, pipeline can be used to perform automatic text summarization using a pre-trained Transformer model. The input text is provided to the summarization pipeline, which processes the text and generates a concise summary.

===== MENU =====
1. Sentiment Analysis
2. Text Generation
3. Summarization
4. Question Answering
5. Translation
6. Exit
Enter your choice: 4
No model was supplied, defaulted to distilbert/distilbert-base-cased-distilled-squad and revision 564e9b5 (https://huggingface.co/distilbert/distilbert-base-cased-distilled-squad).
Using a pipeline without specifying a model name and revision in production is not recommended.
Device set to use mps:0
Enter Context:
Delhi is capital of india.
Enter Question: what is capital of india?

Answer: Delhi
```

```
print("\nModel 2:")
classifier = pipeline(
    "sentiment-analysis",
    model="lxyuan/distilbert-base-multilingual-cased-sentiments-student"
)
result = classifier(text)
print(result)
```

```
def wrong_model():
    print("\nTrying GPT-2 for Sentiment Analysis")
```

```
try:
    classifier = pipeline(
        "sentiment-analysis",
        model="gpt2"
    )
```

```
text = input("Enter text: ")
result = classifier(text)
```

```
print("\nResult:", result)
```

```
except Exception as e:
```

```
print("\nError:", e)
```

```
while True:
```

```
    print("\n===== MENU =====")
```

```
    print("1. Sentiment Analysis")
```

```
    print("2. Text Generation")
```

```
    print("3. Summarization")
```

```
    print("4. Question Answering")
```

```
    print("5. Translation")
```

```
    print("6. Different Models")
```

```
    print("7. Model for Another Task")
```

```
    print("8. Exit")
```

```
choice = input("Enter your choice: ")
```

```
if choice == "1":
```

```
    sentiment_analysis()
```

```
elif choice == "2":
```

```
    text_generation()
```

```
elif choice == "3":
```

```
    summarization()
```

```
elif choice == "4":
```

```
    question_answering()
```

```
elif choice == "5":
```

```
    translation()
```

```
elif choice == "6":
```

```
    different_models()
```

```
elif choice == "7":
```

```
    wrong_model()
```

```
elif choice == "8":
```

```
    print("Exiting...")
```

```
    break
```

```
else:
```

```
    print("Invalid choice!")
```

RESULT

The menu-driven NLP application was successfully implemented using the Transformers pipeline() function.

The application was able to perform:

- 1.Sentiment Analysis
- 2.Text Generation
- 3.Summarization
- 4.Question Answering
- 5.English-to-Hindi Translation

Different models were also tested using the model parameter. A model intended for another task was tested, and GPT-2 text generation was performed using max_new_tokens and temperature.

Output: The respective result is displayed according to the option selected by the user.

LEARNING OUTCOMES

1. Learned how to use the Transformers pipeline() function.
2. Learned how to perform multiple NLP tasks using pre-trained models.
3. Learned how to create a menu-driven NLP application.
4. Learned how pipeline() can automatically select suitable models.
5. Learned how to specify a particular model using the model parameter.
6. Learned how different models can be used for the same NLP task.
7. Understood why a model designed for one task may not work correctly for another task.
8. Learned how max_new_tokens and temperature affect text generation.
9. Gained practical experience with Transformer-based NLP applications.

EXPERIMENT – 2

PROBLEM STATEMENT: Take one/two functions from Experiment 1, such as Sentiment Analysis, and create a Gradio application out of it. The Gradio application has to be deployed on Render / Hugging Face.

OBJECTIVES

1. To create a web-based interface for the Sentiment Analysis function developed in Experiment 1.
2. To understand the basic working of the Gradio library.
3. To connect a Transformer-based machine learning model with a Gradio interface.
4. To accept text input from the user through a web interface.
5. To display the predicted sentiment through the Gradio application.
6. To deploy the Gradio application on a cloud platform such as Render or Hugging Face.

THEORY

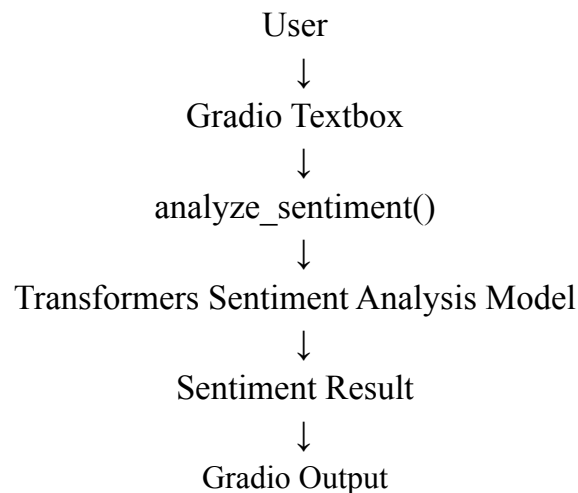
1. Overview of the Experiment

Gradio is a Python library used to create simple web interfaces for machine learning models and Python functions.

In this experiment, the Sentiment Analysis function from Experiment 1 is converted into a web application using Gradio.

The user enters text into a textbox. The text is passed to the Transformer sentiment-analysis pipeline. The model analyzes the text and returns its predicted sentiment.

The basic flow of the application is:



2. Libraries Used

Gradio

Gradio is used to create the web interface for the sentiment analysis application.

`import gradio as gr`

Transformers

The Transformers library is used to load the pre-trained sentiment analysis model through the pipeline() function.

`from transformers import pipeline`

3. Functions Used

pipeline()

The pipeline() function creates a sentiment-analysis pipeline.

```
classifier = pipeline("sentiment-analysis")
```

analyze_sentiment()

This user-defined function receives the text entered by the user, passes it to the classifier, and returns the predicted sentiment.

```
def analyze_sentiment(text):  
    result = classifier(text)  
    return result[0]["label"]
```

gr.Interface()

gr.Interface() connects the Python function with the input and output components of the web interface.

```
app = gr.Interface(  
    fn=analyze_sentiment,  
    inputs=gr.Textbox(label="Enter the Sentiment"),  
    outputs=gr.Textbox(label="Analyzed Sentiment result")  
)
```

app.launch()

The launch() function starts the Gradio application.

```
app.launch(share=True)
```

The share=True option creates a shareable link for the running Gradio application.

WORKING OF PROGRAM

- The program first imports Gradio and the Transformers pipeline() function.

```
import gradio as gr  
from transformers import pipeline
```

- A sentiment-analysis classifier is created:

```
classifier = pipeline("sentiment-analysis")
```

- The analyze_sentiment() function receives the text entered by the user.

```
def analyze_sentiment(text):  
    if len(text) == 0:  
        return "Please enter some text"
```

```
    result = classifier(text)  
    return result[0]["label"]
```

If the user does not enter any text, the application displays a message asking the user to enter text. Otherwise, the text is passed to the sentiment-analysis model.

- The predicted label is extracted using:

```
result[0]["label"]
```

- The Gradio interface is then created:

```
app = gr.Interface(  
    fn=analyze_sentiment,  
    inputs=gr.Textbox(label="Enter the Sentiment"),  
    outputs=gr.Textbox(label="Analyzed Sentiment result")  
)
```

- Finally, the application is launched:

```
app.launch(share=True)
```

PROGRAM CODE

```
import gradio as gr  
from transformers import pipeline  
  
classifier = pipeline("sentiment-analysis")  
  
def analyze_sentiment(text):  
  
    if len(text) == 0:  
        return "Please enter some text"  
  
    result = classifier(text)  
    return result[0]["label"]  
  
app = gr.Interface(  
    fn=analyze_sentiment,  
    inputs=gr.Textbox(label="Enter the Sentiment"),  
    outputs=gr.Textbox(label="Analyzed Sentiment result")  
)  
  
app.launch(share=True)
```

DEPLOYMENT

1. The Gradio application is required to be deployed on Render or Hugging Face as specified in the experiment.
2. For deployment, the project should contain the Python application and its required dependencies.
3. A requirements.txt file can contain:
 - gradio
 - transformers
 - torch
 - sentencepiece
4. The application file contains the Gradio code and is executed by the deployment platform.
5. After successful deployment, the application can be accessed through the URL provided by the hosting platform.

Output: Gradio based application

[transformers] No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f.
Using a pipeline without specifying a model name and revision in production is not recommended.

Loading weights: 100%  104/104 [00:00<00:00, 3752.74it/s]

* Running on local URL: <http://127.0.0.1:7862>

* Running on public URL: <https://6d6a3b0cce4e3dfcbc.gradio.live>

This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces (<https://huggingface.co/spaces>)

Enter the Sentiment good movie	Analyzed Sentiment result POSITIVE	
Clear	Submit	Flag

RESULT

A Gradio-based Sentiment Analysis web application was successfully created using the Sentiment Analysis function from Experiment 1.

The application accepts text from the user and displays the predicted sentiment as the output.

Example:

Input:

I really enjoyed this movie.

Output:

POSITIVE

The application can also be deployed on Render or Hugging Face according to the experiment requirement.

LEARNING OUTCOMES

After completing this experiment:

1. Learned how to create a simple web interface using Gradio.
2. Learned how to connect a Transformer model with a Gradio application.
3. Learned how to accept user input through a web interface.
4. Learned how to display machine learning predictions through Gradio.
5. Learned how to reuse the Sentiment Analysis function from Experiment 1.
6. Learned the basic process of deploying a machine learning application on a cloud platform.
7. Understood how a Python-based machine learning function can be converted into an interactive web application.

EXPERIMENT – 3

PROBLEM STATEMENT

Perform the following Natural Language Processing tasks:

- 1.Perform Tokenization using spaCy (Classical NLP).
- 2.Perform Tokenization using tokenizers of popular modern NLP models and compare the tokenization outputs of BERT, GPT, T5, and DeepSeek on the same input.
- 3.Perform Tokenization using spaCy after reading text from a PDF file and a DOCX file.
- 4.Write a program to take a paragraph as input and remove the stopwords using spaCy.
- 5.Test different attributes of a spaCy Token including lower_, shape_, prefix_, suffix_, is_alpha, is_digit, is_numeric, is_punct, is_space, is_title, like_email, like_url, i, idx, n_lefts, and n_rights.

OBJECTIVES

- 1.To understand tokenization in Natural Language Processing.
- 2.To perform classical NLP tokenization using spaCy.
- 3.To understand how modern Transformer models tokenize text.
- 4.To compare the tokenization behavior of BERT, GPT-2, T5, and DeepSeek.
- 5.To extract text from PDF and DOCX files and perform tokenization.
- 6.To understand and remove stopwords using spaCy.
- 7.To study different attributes of spaCy Token objects.
- 8.To understand the difference between classical NLP tokenization and modern Transformer tokenization.

THEORY

1. Overview of the Experiment

Tokenization

Tokenization is the process of breaking text into smaller units called **tokens**. A token can be a word, punctuation mark, number, or a subword depending on the tokenizer being used.

For example:

Input:Natural Language Processing is amazing!

Output:

Natural
Language
Processing
is
amazing
!

Tokenization is an important preprocessing step in NLP because machine learning models process text through numerical representations of tokens.

spaCy Tokenization

spaCy is a Python library used for Natural Language Processing. Its tokenizer divides text into tokens and provides several useful linguistic attributes for each token.

Example:

```
doc = nlp(text)
for token in doc:
    print(token.text)
```

Transformer Tokenization

Modern Transformer models use their own tokenization methods. Different models can divide the same sentence into different tokens.

In this experiment, tokenization is compared for:

- BERT
- GPT-2
- T5
- DeepSeek

The AutoTokenizer class from Transformers is used to load the appropriate tokenizer for each model.

Stopwords

Stopwords are commonly occurring words that may carry less useful information for some NLP tasks.

Examples include:

the
is
a
an
and
of
to

spaCy provides the `is_stop` attribute to identify stopwords.

spaCy Token Attributes

spaCy provides many attributes that describe individual tokens.

`lower_`: Lowercase form of the token

`shape_`: Shape pattern of the token

`prefix_`: Prefix of the token

`suffix_`: Suffix of the token

`is_alpha`: Checks whether the token contains alphabetic characters

`is_digit`: Checks whether the token consists of digits

`is_numeric`: Checks whether the token represents a number

`is_punct`: Checks whether the token is punctuation

`is_space`: Checks whether the token is whitespace

`is_title`: Checks whether the token is title-cased

`like_email`: Checks whether the token resembles an email address

like_url: Checks whether the token resembles a URL

i: Position/index of the token in the document

idx: Character offset of the token in the original text

n_lefts: Number of left dependency children

n_rights: Number of right dependency children

These are the attributes specified in the experiment.

2. LIBRARIES USED

spaCy

Used for classical NLP tokenization, stopwords removal, and testing token attributes.

```
import spacy
```

Transformers

Used for loading the tokenizers of BERT, GPT-2, T5, and DeepSeek.

```
from transformers import AutoTokenizer
```

PyPDF

Used to read and extract text from PDF files.

```
from pypdf import PdfReader
```

python-docx

Used to read and extract text from DOCX files.

```
from docx import Document
```

The required libraries can be installed using:

```
!pip install spacy transformers torch pypdf python-docx
```

```
!python -m spacy download en_core_web_sm
```

3. FUNCTIONS USED

spacy.load()

Loads the spaCy English language model.

```
nlp = spacy.load("en_core_web_sm")
```

nlp()

Processes input text and creates a spaCy Doc object.

```
doc = nlp(text)
```

AutoTokenizer.from_pretrained()

Loads the tokenizer associated with a particular Transformer model.

```
tokenizer = AutoTokenizer.from_pretrained(model_id)
```

tokenize()

Converts text into tokens using the selected Transformer tokenizer.

```
tokens = tokenizer.tokenize(text)
```

convert_tokens_to_ids()

Converts the generated tokens into their corresponding numerical token IDs.

```
tokenizer.convert_tokens_to_ids(tokens)
```

PdfReader()

Reads a PDF document.

```
reader = PdfReader("sample.pdf")
```

Document()

Reads a DOCX document.

```
document = Document("sample.docx")
```

input()

Accepts a paragraph from the user.

print()

Displays tokens, extracted text, and token attributes.

WORKING OF PROGRAM

The experiment is divided into five parts.

(i) Tokenization using spaCy

First, the English spaCy model is loaded:

```
nlp = spacy.load("en_core_web_sm")
```

The input text is processed:

```
doc = nlp(text)
```

The program then iterates through the document and prints every token.

for token in doc:

```
    print(token.text)
```

(ii) Tokenization using BERT, GPT-2, T5 and DeepSeek

The AutoTokenizer class is used to load the tokenizer for each model.

The models used are:

```
models = {  
    "BERT": "bert-base-uncased",  
    "GPT-2": "gpt2",  
    "T5": "t5-small",  
    "DeepSeek": "deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B"  
}
```

The same input text is passed to each tokenizer.

```
tokens = tokenizer.tokenize(text)
```

The tokens and their corresponding token IDs are displayed.

This allows the tokenization outputs of different modern NLP models to be compared.

(iii) Tokenization from PDF and DOCX

A PDF file is opened using:

```
reader = PdfReader("sample.pdf")
```

Text is extracted from each page.

Similarly, the DOCX file is opened using:

```
document = Document("sample.docx")
```

Text is extracted from each paragraph.

The extracted text from both files is then passed to spaCy:

```
doc = nlp(text)
```

and:

```
docx_doc = nlp(docx_text)
```

The tokens are then printed.

(iv) Removing Stopwords

The user enters a paragraph:

```
text = input("Enter a paragraph: ")
```

The paragraph is processed using spaCy.

Each token is checked using:

```
if not token.is_stop:
```

Only tokens that are not stopwords are added to the result.

The program then prints the paragraph after removing stopwords.

The program also demonstrates removal of both stopwords and punctuation:

```
if not token.is_stop and not token.is_punct:
```

(v) Testing spaCy Token Attributes

A sample sentence containing normal words, a URL, an email address, and a number is processed.

For every token, the program displays the required attributes.

The missing `is_numeric` attribute has been added:

```
print("is_numeric :", token.is_numeric)
```

Tokenization using spaCy

Original Text:
Natural Language Processing is an important field of Artificial Intelligence.

Tokens:
Natural
Language
Processing
is
an
important
field
of
Artificial
Intelligence
.

Tokenization using BERT, GPT-2, T5 and DeepSeek

```
=====
BERT
=====
Tokens:
['natural', 'language', 'processing', 'is', 'amazing', '!']

Token IDs:
[3019, 2653, 6364, 2003, 6429, 999]

=====
GPT-2
=====
Tokens:
['Natural', 'GLanguage', 'GProcessing', 'Gis', 'Gamazing', '!']

Token IDs:
[35364, 15417, 28403, 318, 4998, 0]

=====
T5
=====
Tokens:
['_Natural', '_Language', '_Processing', '_is', '_amazing', '!']

Token IDs:
[6869, 10509, 19125, 19, 1237, 55]

=====
DeepSeek
=====
Tokens:
['Natural', 'GLanguage', 'GProcessing', 'Gis', 'Gamazing', '!']

Token IDs:
[54281, 11434, 28125, 374, 7897, 0]
```

PROGRAM CODE

```
!pip install spacy transformers torch pypdf python-docx
!python -m spacy download en_core_web_sm
```

(i) Tokenization using spaCy

```
import spacy
nlp = spacy.load("en_core_web_sm")

text = "Natural Language Processing is an important field of Artificial Intelligence."
doc = nlp(text)

print("Original Text:")
print(text)

print("\nTokens:")
for token in doc:
    print(token.text)
```

#(ii) Tokenization using BERT, GPT, T5 and DeepSeek

```
from transformers import AutoTokenizer
text = "Natural Language Processing is amazing!"

models = {
    "BERT": "bert-base-uncased",
    "GPT-2": "gpt2",
    "T5": "t5-small",
    "DeepSeek": "deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B"
}

for model_name, model_id in models.items():
    print("\n" + "=" * 50)
    print(model_name)
    print("=" * 50)

    tokenizer = AutoTokenizer.from_pretrained(model_id)
    tokens = tokenizer.tokenize(text)
    print("Tokens:")
    print(tokens)
    print("\nToken IDs:")
    print(tokenizer.convert_tokens_to_ids(tokens))
```

Tokenization from PDF and DOCX

Ignoring wrong pointing object 12 0 (offset 0)

Ignoring wrong pointing object 13 0 (offset 0)

Extracted Text from PDF:

Lorem ipsum

Vestibulum neque massa, scelerisque sit amet ligula eu, congue molestie mi. Praesent ut varius sem.,

Tokens from PDF:

Lorem

ipsum

Vestibulum

neque

massa

,

scelerisque

sit

amet

ligula

eu

,

congue

molestie

mi

.

Praesent

ut

varius

sem

.

,

Extracted Text from word:

Lorem ipsum ,Vestibulum neque massa, scelerisque sit amet ligula eu, congue molestie mi. Praesent ut varius sem.,

Tokens from Word:

Lorem

ipsum

,

Vestibulum

neque

massa

,

scelerisque

sit

amet

ligula

eu

,

congue

molestie

mi

.

Praesent

ut

varius

sem

.

(iii) Extract and read text from PDF and DOCX

```
import spacy
from pypdf import PdfReader
from docx import Document

nlp = spacy.load("en_core_web_sm")

reader = PdfReader("sample.pdf")
document = Document("sample.docx")

text = ""
docx_text = ""
for page in reader.pages:
    page_text = page.extract_text()

    if page_text:
        text += page_text + "\n"

print("Extracted Text from PDF:")
print(text)
doc = nlp(text)
print("\nTokens from PDF:")
for token in doc:
    print(token.text)

for paragraph in document.paragraphs:
    docx_text += paragraph.text + "\n"

print("Extracted Text from Word:")
print(docx_text)

docx_doc = nlp(docx_text)
print("\nTokens from Word:")
for token in docx_doc:
    print(token.text)

#(iv) Remove Stopwords using spaCy

import spacy
nlp = spacy.load("en_core_web_sm")
text = input("Enter a paragraph: ")
doc = nlp(text)
```

```
print("\nOriginal Paragraph:")
print(text)
print("\nAfter Removing Stopwords:")
result = []
```

```
for token in doc:
    if not token.is_stop:
        result.append(token.text)
```

```
print(" ".join(result))
```

```
# Remove punctuation along with stopwords
```

```
import spacy
nlp = spacy.load("en_core_web_sm")
text = input("Enter a paragraph: ")
doc = nlp(text)
result = []
for token in doc:
    if not token.is_stop and not token.is_punct:
        result.append(token.text)
```

```
print("\nAfter Removing Stopwords and Punctuation:")
print(" ".join(result))
```

```
 #(v) Test spaCy Token Attributes
```

```
import spacy
nlp = spacy.load("en_core_web_sm")
text = "Hello Keshav! Visit https://example.com or email me at test@example.com. I have 25 apples."
doc = nlp(text)
for token in doc:
    print("=" * 50)
    print("Token      :", token.text)
    print("lower_     :", token.lower_)
    print("shape_      :", token.shape_)
    print("prefix_     :", token.prefix_)
    print("suffix_     :", token.suffix_)

    print("is_alpha    :", token.is_alpha)
    print("is_digit    :", token.is_digit)
    print("is_numeric  :", token.is_numeric)
    print("is_punct    :", token.is_punct)
    print("is_space    :", token.is_space)
    print("is_title    :", token.is_title)
```

```
print("like_email :", token.like_email)
print("like_url   :", token.like_url)

print("i          :", token.i)
print("idx        :", token.idx)

print("n_lefts    :", token.n_lefts)
print("n_rights   :", token.n_rights)
```

RESULT

The following NLP operations were successfully implemented:

1. Tokenization using spaCy was performed successfully.
2. Tokenization using BERT, GPT-2, T5, and DeepSeek tokenizers was performed and compared.
3. Text was successfully extracted from PDF and DOCX files and tokenized using spaCy.
4. Stopwords were successfully removed from a user-provided paragraph.
5. Stopwords and punctuation were also removed from the paragraph.
6. The required spaCy Token attributes were successfully tested, including `is_numeric`.

The tokenization output and token IDs were displayed for the selected Transformer models.

LEARNING OUTCOMES

After completing this experiment:

- 1.Learned the concept of tokenization in NLP.
- 2.Learned how to perform tokenization using spaCy.
- 3.Learned how Transformer models use different tokenizers.
- 4.Compared tokenization outputs of BERT, GPT-2, T5, and DeepSeek.
- 5.Learned how to extract text from PDF and DOCX files.
- 6.Learned how to tokenize extracted document text using spaCy.
- 7.Learned how to identify and remove stopwords.
- 8.Learned how to remove punctuation along with stopwords.
- 9.Learned about different attributes available in a spaCy Token.
- 10.Understood how token properties such as `is_alpha`, `is_digit`, `is_numeric`, `like_email`, `like_url`, `idx`, `n_lefts`, and `n_rights` can be examined.

Test spaCy Token Attributes

```
=====
Token      : Hello
lower_     : hello
shape_     : Xxxxx
prefix_    : H
suffix_    : llo
is_alpha   : True
is_digit   : False
is_punct   : False
is_space   : False
is_title   : True
like_email : False
like_url   : False
i          : 0
idx        : 0
n_lefts    : 0
n_rights   : 0
=====
Token      : Keshav
lower_     : keshav
shape_     : Xxxxx
prefix_    : K
suffix_    : hav
is_alpha   : True
is_digit   : False
is_punct   : False
is_space   : False
is_title   : True
like_email : False
like_url   : False
i          : 1
idx        : 6
n_lefts    : 1
n_rights   : 1
=====
Token      : !
lower_     : !
shape_     : !
prefix_    : !
suffix_    : !
is_alpha   : False
is_digit   : False
is_punct   : True
is_space   : False
is_title   : False
like_email : False
like_url   : False
i          : 2
idx        : 12
n_lefts    : 0
n_rights   : 0
=====
```

EXPERIMENT 4

Problem Statement

Develop a production-ready sentiment analysis service using a pre-trained Transformer model. The system should provide both a Gradio-based user interface and a FastAPI REST API. The sentiment model should be implemented only once and reused by both interfaces.

Objective

- To perform sentiment analysis using a pre-trained Transformer model.
- To create an interactive Gradio interface.
- To create a REST API using FastAPI.
- To reuse the same prediction function in both applications.
- To understand the basic architecture of a machine-learning service.

Theory

Sentiment analysis is an NLP task used to determine the emotional polarity of text. It generally classifies text into categories such as positive or negative.

The Hugging Face Transformers library provides pre-trained models that can perform sentiment analysis without training a model from scratch. In this experiment, the `distilbert-base-uncased-finetuned-sst-2-english` model is used.

The application is divided into three components:

- `model.py` — contains the sentiment-analysis model and prediction function.
- `gradio_app.py` — provides an interactive web interface.
- `api.py` — provides a REST API using FastAPI.

Both Gradio and FastAPI call the same `predict_sentiment()` function from `model.py`.

Libraries Used

- Transformers
- Gradio
- FastAPI
- Pydantic
- Python

Algorithm / Working

1. Load the pre-trained DistilBERT sentiment-analysis model.
2. Define a `predict_sentiment()` function.
3. Pass the input text to the Transformer model.
4. Obtain the sentiment label and confidence score.
5. Use the prediction function in the Gradio application.
6. Use the same prediction function in the FastAPI application.
7. Display the result to the user.

Project Structure

SENTIMENTAL_SERVICES/

```
|— model.py  
|— gradio_app.py  
|— api.py
```

Code

model.py

```
from transformers import pipeline
```

```
classifier = pipeline(  
    "sentiment-analysis",  
    model="distilbert-base-uncased-finetuned-sst-2-english"  
)
```

```
def predict_sentiment(text):  
    result = classifier(text)[0]  
    return {  
        "label": result["label"],  
        "confidence": float(result["score"])  
    }
```

gradio_app.py

```
import gradio as gr  
from model import predict_sentiment
```

```
def predict(text):  
    result = predict_sentiment(text)  
    return f'{result["label"]} ({result["confidence"]:.3f})'
```

```
demo = gr.Interface(  
    fn=predict,  
    inputs="text",  
    outputs="text"  
)
```

```
demo.launch()
```

api.py

```
from fastapi import FastAPI
from pydantic import BaseModel
from model import predict_sentiment
```

```
app = FastAPI()
```

```
class TextInput(BaseModel):
    text: str
```

```
@app.get("/")
def home():
    return {
        "message": "Sentiment Analysis API is running"
    }
```

```
@app.post("/predict")
def predict(data: TextInput):
    result = predict_sentiment(data.text)
    return result
```

Sample Input

I really enjoyed this movie.

Sample Output

POSITIVE (0.999)

The FastAPI /predict endpoint returns a JSON response containing the sentiment label and confidence score.

```
{
  "label": "POSITIVE",
  "confidence": 0.99
}
```

default

GET / Home

POST /predict Predict

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "text": "I love this movie"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "text": "I love this movie"
  }'
```

Request URL

<http://127.0.0.1:8000/predict>

Server response

Code Details

200

Response body

```
{
  "label": "POSITIVE",
  "confidence": 0.9998766183853149
}
```

Response headers

```
content-length: 52
content-type: application/json
date: Mon, 07 Sep 2026 04:25:09 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links
	Media type application/json	
	Controls Accept header.	
	Example Value Schema	
	"string"	
422	Validation Error	No links
	Media type application/json	
	Example Value Schema	
	<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	

Result

The sentiment analysis service was successfully implemented using a pre-trained DistilBERT model. Both the Gradio interface and FastAPI REST API successfully use the same sentiment prediction function.

Learning Outcomes

- Learned how to use a pre-trained Transformer model.
- Learned how to build an NLP inference service.
- Learned how to create a Gradio interface.
- Learned how to create a FastAPI REST API.
- Understood model reuse and modular application architecture.

EXPERIMENT 5

Problem Statement

Prepare an interactive dashboard that provides an interface for performing different Natural Language Processing operations, including Named Entity Recognition, POS Tagging, POS Distribution, Lemmatization, Stemming, Morphology, and Dependency Parsing.

Objective

- To perform different NLP operations on user-provided text.
- To understand linguistic properties of words and sentences.
- To build an interactive NLP dashboard using Gradio.
- To visualize dependency relationships between words.

Theory

Natural Language Processing provides several techniques for analyzing the structure and meaning of text.

Named Entity Recognition (NER) identifies entities such as people, organizations, locations, dates, and other named entities.

Part-of-Speech (POS) Tagging assigns grammatical categories such as noun, verb, adjective, and adverb to individual tokens.

POS Distribution counts the occurrence of different POS categories in the input text.

Lemmatization converts words into their base or dictionary form.

Stemming reduces words to their root-like form using rules. In this experiment, the Porter Stemmer is used.

Morphology describes grammatical features of tokens, such as tense, number, gender, person, and degree where applicable.

Dependency Parsing identifies grammatical relationships between words in a sentence. spaCy's displacy is used to display the dependency tree.

Libraries Used

- spaCy
- Gradio
- NLTK
- Pandas
- Python

Algorithm / Working

1. Load the en_core_web_sm spaCy model.
2. Accept text from the user.
3. Process the text using spaCy.
4. Perform the selected NLP operation.
5. Display the result in the corresponding dashboard tab.
6. Use spaCy's displacy to visualize dependency relationships.

NLP Operations Implemented

1. Named Entity Recognition

The input text is processed using spaCy and detected entities are displayed along with their labels.

2. POS Tagging

Each token is displayed with its POS category and detailed grammatical tag.

3. POS Distribution

The frequency of each POS category is calculated using Counter.

4. Lemmatization

Each token is mapped to its lemma using spaCy.

5. Stemming

The Porter Stemmer from NLTK is used to generate stems.

6. Morphology

The morphological features associated with each token are displayed.

7. Dependency Parsing

spaCy's `displacy.render()` with `style="dep"` is used to display the dependency tree.

Code

```
import gradio as gr
import spacy
import pandas as pd
from nltk.stem import PorterStemmer
from collections import Counter
from spacy import displacy

nlp = spacy.load("en_core_web_sm")

stemmer = PorterStemmer()

def named_entities(text):
    doc = nlp(text)

    data = []

    for ent in doc.ents:
        data.append({
            "Entity": ent.text,
            "Label": ent.label_
        })
    if data:
        return pd.DataFrame(data)

    return pd.DataFrame(columns=["Entity", "Label"])
```

```
def pos_tagging(text):
    doc = nlp(text)

    data = []

    for token in doc:
        if not token.is_space:
            data.append({
                "Token": token.text,
                "POS": token.pos_,
                "Tag": token.tag_
            })

    return pd.DataFrame(data)


def pos_distribution(text):
    doc = nlp(text)

    pos_counts = Counter(
        token.pos_
        for token in doc
        if not token.is_space
    )

    data = [
        {
            "POS": pos,
            "Count": count
        }
        for pos, count in pos_counts.items()
    ]

    return pd.DataFrame(data).sort_values(
        by="Count",
        ascending=False
    )
```

```
def lemmatization(text):
    doc = nlp(text)

    data = []

    for token in doc:
        if not token.is_space:
            data.append({
                "Token": token.text,
                "Lemma": token.lemma_
            })

    return pd.DataFrame(data)


def stemming(text):
    doc = nlp(text)

    data = []

    for token in doc:
        if not token.is_space and token.is_alpha:
            data.append({
                "Token": token.text,
                "Stem": stemmer.stem(token.text)
            })

    return pd.DataFrame(data)


def morphology(text):
    doc = nlp(text)

    data = []

    for token in doc:
        if not token.is_space:
            data.append({
                "Token": token.text,
                "Morphology": str(token.morph)
            })
    return pd.DataFrame(data)
```

```
def dependency_parsing(text):  
    doc = nlp(text)
```

```
    html = displacy.render(  
        doc,  
        style="dep",  
        jupyter=False  
    )
```

```
    return html
```

```
with gr.Blocks(title="NLP Dashboard") as demo:
```

```
    gr.Markdown(  
        """  
        # NLP Dashboard  
  
        Enter a sentence or paragraph and select an NLP operation.  
        """  
    )
```

```
    text_input = gr.Textbox(  
        label="Enter Text",  
        placeholder="Enter your sentence here...",  
        lines=5  
    )
```

```
    with gr.Tab("Named Entity Recognition"):  
        ner_button = gr.Button("Perform NER")  
        ner_output = gr.Dataframe(  
            headers=["Entity", "Label"],  
            label="Named Entities"  
        )
```

```
        ner_button.click(  
            fn=named_entities,  
            inputs=text_input,  
            outputs=ner_output  
        )
```

```
    with gr.Tab("POS Tagging"):
```

```
pos_button = gr.Button("Perform POS Tagging")
pos_output = gr.Dataframe(
    headers=["Token", "POS", "Tag"],
    label="POS Tags"
)

pos_button.click(
    fn=pos_tagging,
    inputs=text_input,
    outputs=pos_output
)

with gr.Tab("POS Distribution"):
    distribution_button = gr.Button("Calculate POS Distribution")
    distribution_output = gr.Dataframe(
        headers=["POS", "Count"],
        label="POS Distribution"
    )

    distribution_button.click(
        fn=pos_distribution,
        inputs=text_input,
        outputs=distribution_output
    )

with gr.Tab("Lemmatization"):
    lemma_button = gr.Button("Perform Lemmatization")
    lemma_output = gr.Dataframe(
        headers=["Token", "Lemma"],
        label="Lemmatization"
    )

    lemma_button.click(
        fn=lemmatization,
        inputs=text_input,
        outputs=lemma_output
    )

with gr.Tab("Stemming"):
    stem_button = gr.Button("Perform Stemming")
    stem_output = gr.Dataframe(
        headers=["Token", "Stem"],
```

```
        label="Stemming"
    )

    stem_button.click(
        fn=stemming,
        inputs=text_input,
        outputs=stem_output
    )

with gr.Tab("Morphology"):
    morph_button = gr.Button("Analyze Morphology")
    morph_output = gr.DataFrame(
        headers=["Token", "Morphology"],
        label="Morphological Features"
    )

    morph_button.click(
        fn=morphology,
        inputs=text_input,
        outputs=morph_output
    )

with gr.Tab("Dependency Parsing"):
    dependency_button = gr.Button("Show Dependencies")
    dependency_output = gr.HTML(
        label="Dependency Tree"
    )

    dependency_button.click(
        fn=dependency_parsing,
        inputs=text_input,
        outputs=dependency_output
    )

demo.launch()
```

Name Entity Recognition

NLP Dashboard

Enter a sentence or paragraph and select an NLP operation.

Enter Text

i Love this movie.

Named Entity Recognition

POS Tagging

POS Distribution

Lemmatization

Stemming

Morphology

Dependency Parsing

Perform NER

Named Entities

Entity	Label
Love	WORK_OF_ART

POS Tagging

Named Entity Recognition

POS Tagging

POS Distribution

Lemmatization

Stemming

Morphology

Dependency Parsing

Perform POS Tagging

POS Tags

Token	POS	Tag
i	PRON	PRP
Love	VERB	VBP
this	DET	DT
movie	NOUN	NN
.	PUNCT	.

POS Distribution

Named Entity Recognition

POS Tagging

POS Distribution

Lemmatization

Stemming

Morphology

Dependency Parsing

Calculate POS Distribution

POS Distribution

POS	Count
PRON	1
VERB	1
DET	1
NOUN	1
PUNCT	1

Lemmatization

[Named Entity Recognition](#)[POS Tagging](#)[POS Distribution](#)[Lemmatization](#)[Stemming](#)[Morphology](#)[Dependency Parsing](#)

Perform Lemmatization

Lemmatization

Token	Lemma
i	I
Love	love
this	this
movie	movie
.	.

Stemming

[Named Entity Recognition](#)[POS Tagging](#)[POS Distribution](#)[Lemmatization](#)[Stemming](#)[Morphology](#)[Dependency Parsing](#)

Perform Stemming

Stemming

Token	Stem
i	i
Love	love
this	thi
movie	movi

[Use via API](#)  · [Built with Gradio](#)  · [Settings](#) 

Morphology

[Named Entity Recognition](#)[POS Tagging](#)[POS Distribution](#)[Lemmatization](#)[Stemming](#)[Morphology](#)[Dependency Parsing](#)

Analyze Morphology

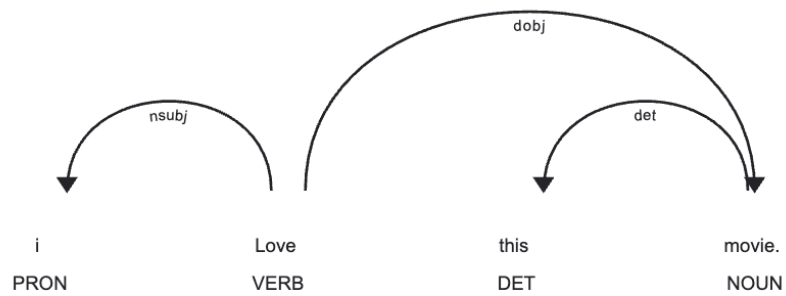
Morphological Features

Token	Morphology
i	Case=Nom Number=Sing Person=1 PronType=Prs
Love	Tense=Pres VerbForm=Fin
this	Number=Sing PronType=Dem
movie	Number=Sing
.	PunctType=Peri

[Use via API](#)  · [Built with Gradio](#)  · [Settings](#) 

Dependency Parsing

[Named Entity Recognition](#)[POS Tagging](#)[POS Distribution](#)[Lemmatization](#)[Stemming](#)[Morphology](#)[Dependency Parsing](#)**Show Dependencies**

Sample Input

Apple is planning to open a new office in London next year.

Sample Output

The dashboard displays:

- Detected entities such as Apple and London
- POS tags for each token
- POS frequency distribution
- Token and lemma pairs
- Token and stem pairs
- Morphological features
- A dependency tree showing grammatical relationships

Result

An interactive NLP dashboard was successfully created. The dashboard performs all seven required NLP operations on user-provided text.

Learning Outcomes

- Learned Named Entity Recognition.
- Learned POS tagging and POS distribution.
- Understood the difference between stemming and lemmatization.
- Learned how to analyze morphological features.
- Learned dependency parsing and dependency visualization.
- Learned how to build an interactive NLP dashboard using Gradio.

EXPERIMENT 6

Problem Statement

Perform different text-processing operations using Python and machine-learning libraries. The experiment includes creation of Bag of Words and TF-IDF matrices, calculation of cosine similarity between two documents with common-token extraction, and generation of all n-grams from a sufficiently long sentence.

Part (i) — Bag of Words and TF-IDF

Objective

- To create a Bag of Words matrix using Scikit-learn.
- To create a TF-IDF matrix using Scikit-learn.
- To display the matrices in tabular format.

Theory

Bag of Words

Bag of Words represents documents using the frequency of words occurring in each document. Each unique word becomes a feature, and the corresponding value represents its frequency.

TF-IDF

TF-IDF stands for Term Frequency-Inverse Document Frequency. It gives higher importance to words that are frequent in a document but less frequent across the complete collection of documents.

Code

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
import pandas as pd
```

```
documents = [
    "Natural Language Processing is interesting",
    "Natural Language Processing is useful",
    "Machine Learning is interesting"
]
```

```
bow_vectorizer = CountVectorizer()
```

```
bow_matrix = bow_vectorizer.fit_transform(documents)
```

```
bow_df = pd.DataFrame(
    bow_matrix.toarray(),
    columns=bow_vectorizer.get_feature_names_out()
)
```

```
print("BAG OF WORDS MATRIX:")
print(bow_df)
```

```
tfidf_vectorizer = TfidfVectorizer()
```

```
tfidf_matrix = tfidf_vectorizer.fit_transform(documents)
```

```
tfidf_df = pd.DataFrame(
    tfidf_matrix.toarray(),
    columns=tfidf_vectorizer.get_feature_names_out()
)
```

```
print("\nTF-IDF MATRIX:")
print(tfidf_df)
```

BAG OF WORDS

BAG OF WORDS MATRIX:

	interesting	is	language	learning	machine	natural	processing	useful
0	1	1	1	0	0	1	1	0
1	0	1	1	0	0	1	1	1
2	1	1	0	1	1	0	0	0

TF-IDF

TF-IDF MATRIX:

	interesting	is	language	learning	machine	natural	processing	useful
0	0.466096	0.361965	0.466096	0.000000	0.000000	0.466096	0.466096	0.000000
1	0.000000	0.336315	0.433067	0.000000	0.000000	0.433067	0.433067	0.569431
2	0.444514	0.345205	0.000000	0.584483	0.584483	0.000000	0.000000	0.000000

Result

Bag of Words and TF-IDF matrices were successfully generated and displayed in tabular format.

Part (ii) — Cosine Similarity and Common Tokens**Objective**

- To accept two files from the user.
- To support PDF, DOCX, and TXT files.
- To extract text from both files.
- To perform preprocessing using stopwords removal and lemmatization.
- To calculate cosine similarity.
- To identify tokens common to both documents.

Theory

Cosine similarity measures the similarity between two documents by calculating the cosine of the angle between their vector representations.

A similarity score close to **1** indicates that the documents are highly similar, while a score close to **0** indicates low similarity.

The documents are converted into TF-IDF vectors before calculating cosine similarity.

Code

```
import os
import spacy
from pypdf import PdfReader
from docx import Document
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

nlp = spacy.load("en_core_web_sm")

def read_file(file_path):
    extension = os.path.splitext(file_path)[1].lower()

    if extension == ".pdf":
        reader = PdfReader(file_path)
        text = ""

        for page in reader.pages:
            page_text = page.extract_text()

            if page_text:
                text += page_text + "\n"

        return text

    elif extension == ".docx":
```

```
document = Document(file_path)
text = ""

for paragraph in document.paragraphs:
    text += paragraph.text + "\n"

return text

elif extension == ".txt":
    with open(file_path, "r", encoding="utf-8") as file:
        return file.read()

else:
    raise ValueError(
        "Unsupported file format. Use PDF, DOCX or TXT."
    )

def preprocess(text):
    doc = nlp(text)

    tokens = []

    for token in doc:
        if not token.is_stop and not token.is_punct and token.is_alpha:
            tokens.append(token.lemma_.lower())

    return tokens

file1 = input("Enter path of first file: ")
file2 = input("Enter path of second file: ")

text1 = read_file(file1)
text2 = read_file(file2)

print("TEXT FROM FIRST FILE")
print(text1)

print("TEXT FROM SECOND FILE")
print(text2)
```

```
tokens1 = preprocess(text1)
tokens2 = preprocess(text2)

processed_text1 = " ".join(tokens1)
processed_text2 = " ".join(tokens2)

documents = [
    processed_text1,
    processed_text2
]

vectorizer = TfidfVectorizer()

tfidf_matrix = vectorizer.fit_transform(documents)

similarity = cosine_similarity(
    tfidf_matrix[0:1],
    tfidf_matrix[1:2]
)

score = similarity[0][0]

print("COSINE SIMILARITY")
print("Similarity Score:", score)

common_tokens = sorted(
    set(tokens1).intersection(set(tokens2))
)

print("COMMON TOKENS")

if common_tokens:
    print("Number of Common Tokens:", len(common_tokens))
    print()

    for token in common_tokens:
        print(token)
else:
    print("No common tokens found.")
```

Cosine Similarity between two Document

TEXT FROM FIRST FILE:-

Natural Language Processing is a field of Artificial Intelligence.
It enables computers to understand and process human language.
NLP is used for text classification, sentiment analysis, machine translation, and text summarization.
Machine learning and deep learning techniques are commonly used in NLP applications.

TEXT FROM SECOND FILE:-

Natural Language Processing is an important area of Artificial Intelligence.
It allows computers to process and understand human language.
NLP applications include sentiment analysis, text classification, machine translation, and summarization.
Deep learning and machine learning methods are widely used to build NLP systems.

COSINE SIMILARITY:-

Similarity Score: 0.7287063408989913

COMMON TOKENS:-

analysis
application
artificial
classification
computer
deep
human
intelligence
language
learning
machine
natural
nlp
process
processing
sentiment
summarization
text
translation
understand

Result

The program successfully accepts PDF, DOCX, and TXT files, preprocesses their text, calculates cosine similarity using TF-IDF vectors, and displays the common tokens.

Part (iii) — N-Gram Generation

Objective

- To tokenize a sufficiently long sentence.
- To generate all possible n-grams.
- To display unigram, bigram, trigram, and higher-order n-grams.

Theory

An n-gram is a contiguous sequence of n tokens from a sentence.

Examples:

- **1-gram:** one token
- **2-gram:** two consecutive tokens
- **3-gram:** three consecutive tokens

For a sentence containing N tokens, the number of n-grams of size n is:

$$N - n + 1$$

Code

```
import spacy
nlp = spacy.load("en_core_web_sm")
text = input("Enter a sufficiently long sentence: ")
doc = nlp(text)
tokens = []

for token in doc:
    if not token.is_punct and not token.is_space:
        tokens.append(token.text)

print("\nOriginal Sentence:")
print(text)
print("\nTokens:")
print(tokens)
number_of_tokens = len(tokens)
print("ALL N-GRAMS")

for n in range(1, number_of_tokens + 1):
    print(f"\n{n}-GRAMS:")
    for i in range(number_of_tokens - n + 1):
        ngram = tokens[i:i + n]
        print(" ".join(ngram))
```

Original Sentence:-

Natural language processing is an important field of artificial intelligence and machine learning.

Tokens:-

['Natural', 'language', 'processing', 'is', 'an', 'important', 'field', 'of', 'artificial', 'intelligence', 'and', 'machine', 'learning']

ALL N-GRAMS:-

1-GRAMS:-

Natural
language
processing
is
an
important
field
of
artificial
intelligence
and
machine
learning

2-GRAMS:-

Natural language
language processing
processing is
is an
an important
important field
field of
of artificial
artificial intelligence
intelligence and
and machine
machine learning

3-GRAMS:-

Natural language processing
language processing is
processing is an
is an important
an important field
important field of
field of artificial
of artificial intelligence
artificial intelligence and
intelligence and machine
and machine learning

4-GRAMS:-

Natural language processing is
language processing is an
processing is an important
is an important field
an important field of
important field of artificial
field of artificial intelligence
of artificial intelligence and
artificial intelligence and machine
intelligence and machine learning

5-GRAMS:-

Natural language processing is an
language processing is an important
processing is an important field
is an important field of
an important field of artificial
important field of artificial intelligence
field of artificial intelligence and
of artificial intelligence and machine
artificial intelligence and machine learning

6-GRAMS:-

Natural language processing is an important
language processing is an important field
processing is an important field of
is an important field of artificial
an important field of artificial intelligence
important field of artificial intelligence and
field of artificial intelligence and machine
of artificial intelligence and machine learning

7-GRAMS:-

Natural language processing is an important field
language processing is an important field of
processing is an important field of artificial
is an important field of artificial intelligence
an important field of artificial intelligence and
important field of artificial intelligence and machine
field of artificial intelligence and machine learning

8-GRAMS:-

Natural language processing is an important field of language processing is an important field of artificial processing is an important field of artificial intelligence is an important field of artificial intelligence and an important field of artificial intelligence and machine important field of artificial intelligence and machine learning

9-GRAMS:-

Natural language processing is an important field of artificial language processing is an important field of artificial intelligence processing is an important field of artificial intelligence and is an important field of artificial intelligence and machine an important field of artificial intelligence and machine learning

10-GRAMS:-

Natural language processing is an important field of artificial intelligence language processing is an important field of artificial intelligence and processing is an important field of artificial intelligence and machine is an important field of artificial intelligence and machine learning

11-GRAMS:-

Natural language processing is an important field of artificial intelligence and language processing is an important field of artificial intelligence and machine processing is an important field of artificial intelligence and machine learning

12-GRAMS:-

Natural language processing is an important field of artificial intelligence and machine language processing is an important field of artificial intelligence and machine learning

13-GRAMS:-

Natural language processing is an important field of artificial intelligence and machine learning

Result

All possible n-grams from the input sentence were successfully generated and displayed.

Learning Outcome

After completing Experiment 6, the following concepts were understood:

- Bag of Words representation
- TF-IDF representation
- Document similarity
- Cosine similarity
- Text preprocessing
- Lemmatization and stopword removal
- Common-token extraction
- N-gram generation
- Text feature representation using Scikit-learn

EXPERIMENT – 7

PROBLEM STATEMENT

Build a semantic search engine over a collection of documents using sentence embeddings and similarity. Extend the system with an Extractive Question Answering model to extract the most relevant answer from the document retrieved by semantic search.

The system should:

1. Store a collection of at least five documents.
2. Generate true sentence embeddings for the documents.
3. Generate an embedding for the user's query.
4. Calculate cosine similarity between the query and documents.
5. Retrieve the most relevant document.
6. Use an Extractive Question Answering model on the retrieved document.
7. Extract and display the answer relevant to the user's question.

The experiment specifically requires the use of true embeddings instead of Bag-of-Words or TF-IDF, with a minimum of five documents and at least two paragraphs per document.

OBJECTIVES

1. To understand the concept of semantic search.
2. To understand sentence embeddings and their role in semantic similarity.
3. To generate vector representations of documents using a pre-trained embedding model.
4. To generate an embedding representation of a user's query.
5. To calculate cosine similarity between query and document embeddings.
6. To retrieve the most semantically relevant document.
7. To understand the concept of Extractive Question Answering.
8. To extract an answer directly from a relevant document.
9. To combine semantic search and Question Answering into a single NLP application.
10. To understand the difference between document retrieval and answer extraction.

THEORY

1. Overview of the Experiment

Semantic Search is an information retrieval technique that searches for information based on the meaning of the query rather than only matching exact keywords.

In this experiment, a collection of documents is converted into numerical vector representations called embeddings.

The user's question is also converted into an embedding.

The similarity between the question and each document is then calculated using cosine similarity.

The document having the highest similarity score is selected as the most relevant document.

The experiment then extends semantic search with Extractive Question Answering. The selected document is given to a Question Answering model along with the user's question, and the model extracts the relevant answer from the document.

The experiment therefore follows the pipeline:

User Question



Question Embedding



Compare with Document Embeddings



Cosine Similarity



Most Relevant Document



Extractive QA Model



Extracted Answer

The experiment specification describes the same two-stage approach: first finding the relevant document and then extracting the answer from it.

2. Semantic Search

Semantic search attempts to understand the meaning of text.

For example:

Query:

How do computers learn from data?

A document containing:

Machine learning allows computers to learn patterns from data and make predictions.

can be considered relevant even though the exact wording of the query and document is different.

This is possible because both the query and document are represented as semantic embeddings.

The experiment uses:

Embedding + Cosine Similarity

for semantic search.

3. Sentence Embeddings

A sentence embedding is a numerical vector representation of a sentence or document.

For example:

Machine learning allows computers to learn from data.

can be converted into a numerical vector:

[0.214, -0.173, 0.421, ..., 0.087]

The complete vector contains many numerical dimensions.

Texts having similar meanings tend to have similar vector representations.

In this experiment, the Sentence Transformers model:

all-MiniLM-L6-v2

is used to generate sentence embeddings.

4. Cosine Similarity

Cosine similarity is used to measure the similarity between two vectors.

The formula is:

$$\text{CosineSimilarity}(A,B) = \{A \cdot B\} / \{|A| |B|\}$$

where:

(A) = query embedding

(B) = document embedding

(A · B) = dot product of the vectors

(|A|) = magnitude of vector A

(|B|) = magnitude of vector B

A higher cosine similarity value indicates greater similarity between the query and document embeddings.

The document with the highest similarity score is selected as the most relevant document.

5. Extractive Question Answering

Extractive Question Answering is an NLP task where the answer is extracted directly from a given context.

For example:

Context:

Machine learning is a branch of artificial intelligence.

It allows computers to learn patterns from data and make predictions without being explicitly programmed for every task.

Question:

What does machine learning allow computers to do?

Answer:

learn patterns from data and make predictions

The answer is obtained from the supplied context rather than being freely generated.

The PPT describes this stage as using an Extractive QA model after finding the relevant document.

6. Document Collection

The experiment requires:

- At least 5 documents
- At least 2 paragraphs per document
- True embeddings
- Semantic similarity

The example topics given in the experiment include:

1. OSI and TCP/IP Models
2. Physical and Data Link Layer
3. Network Layer and Routing
4. Transport Layer TCP and UDP
5. Application Layer HTTP DNS and FTP

These topics are used for demonstrating the document collection in the experiment.

7. Libraries Used

Sentence Transformers

The sentence-transformers library is used to generate semantic sentence embeddings.

```
from sentence_transformers import SentenceTransformer
```

Transformers

The transformers library is used to load the pre-trained Extractive Question Answering model.

```
from transformers import AutoTokenizer
```

```
from transformers import AutoModelForQuestionAnswering
```

PyTorch

PyTorch is used to execute the Question Answering model.

```
import torch
```

Scikit-learn

Scikit-learn is used to calculate cosine similarity.

```
from sklearn.metrics.pairwise import cosine_similarity
```

8. Functions Used

SentenceTransformer()

Loads the pre-trained sentence embedding model.

```
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
```

encode()

Converts text into numerical embeddings.

```
embedding_model.encode(text)
```

cosine_similarity()

Calculates the cosine similarity between vectors.

```
cosine_similarity(query_embedding, document_embeddings)
```

AutoTokenizer.from_pretrained()

Loads the tokenizer associated with the Question Answering model.

```
AutoTokenizer.from_pretrained(  
    "distilbert-base-cased-distilled-squad"  
)
```

AutoModelForQuestionAnswering.from_pretrained()

Loads the pre-trained extractive Question Answering model.

```
AutoModelForQuestionAnswering.from_pretrained(  
    "distilbert-base-cased-distilled-squad"  
)
```

input()

Takes the question from the user.

```
question = input("Enter your question: ")
```

print()

Displays the semantic search and Question Answering results.

```
print(answer)
```

WORKING OF PROGRAM

The program works in two major stages:

Stage 1 – Semantic Search

The program first creates a collection of documents.

Each document is converted into a sentence embedding using the Sentence Transformer model.

The user then enters a question.

The question is also converted into an embedding.

Cosine similarity is calculated between the question embedding and every document embedding.

The document having the highest similarity score is selected.

Stage 2 – Extractive Question Answering

The selected document is used as the context for the Question Answering model.

The user's question and selected document are supplied to the QA model.

The model identifies the start and end positions of the answer in the context.

The corresponding text is extracted and displayed as the answer.

Therefore:

Documents



Document Embeddings



User Question



Question Embedding



Cosine Similarity



Relevant Document



Extractive QA



Answer

PROGRAM CODE

Installation

The required libraries can be installed using:

```
!pip install sentence-transformers
```

```
!pip install transformers
```

```
!pip install torch
```

```
!pip install scikit-learn
```

Complete Program

```
from sentence_transformers import SentenceTransformer
from transformers import AutoTokenizer, AutoModelForQuestionAnswering
from sklearn.metrics.pairwise import cosine_similarity
import torch
```

STEP 1: CREATE DOCUMENT COLLECTION

```
documents = {
    "OSI and TCP/IP Models": [
        """
        The OSI model is a conceptual framework used to understand how
        communication takes place between different systems in a computer
        network. It consists of seven layers: Physical, Data Link, Network,
        Transport, Session, Presentation, and Application.
        """,
        """
        The TCP/IP model is a practical networking model used for communication
        over the Internet. It consists of Network Access, Internet, Transport,
        and Application layers. Protocols such as IP, TCP, UDP, HTTP, and DNS
        operate within different layers of the TCP/IP model.
        """,
    ],
    "Physical and Data Link Layer": [
        """
        The Physical Layer is the lowest layer of the OSI model and is
        responsible for transmitting raw bits over a physical medium. It deals
        with transmission media, signals, connectors, data rates, and the
        physical transmission of data between network devices.
        """,
    ],
}
```

""""

The Data Link Layer provides reliable communication between directly connected devices. It organizes data into frames and uses MAC addresses to identify devices. It also performs error detection and controls access to the shared communication medium.

""""

],

"Network Layer and Routing": [

""""

The Network Layer is responsible for delivering packets from a source network to a destination network. It uses logical addressing such as IP addresses to identify source and destination devices. IPv4 and IPv6 are important protocols associated with this layer.

""",

""""

Routing is the process of determining a suitable path for packets to travel from the source to the destination. Routers use routing tables and routing algorithms to make forwarding decisions. Routing protocols such as RIP, OSPF, and BGP help exchange information about network paths.

""""

],

"Transport Layer TCP and UDP": [

""""

The Transport Layer provides communication between applications running on different hosts. TCP and UDP are two important Transport Layer protocols. TCP is connection-oriented and provides reliable, ordered delivery of data between communicating systems.

""",

""""

TCP uses acknowledgements, retransmission, and flow control to provide reliable communication. UDP is a connectionless transport protocol with lower overhead and does not provide the same reliability mechanisms as TCP. The choice between TCP and UDP depends on the requirements of the application.

""""

],

"Application Layer HTTP DNS and FTP": [

""

The Application Layer provides network services directly to user applications. HTTP is used for communication between web clients and web servers. HTTPS provides secure communication for web applications by using HTTP with encryption.

""

,

""

DNS stands for Domain Name System and is used to translate domain names into IP addresses. FTP is a protocol used for transferring files between computers over a network. HTTP, DNS, and FTP provide important services to network applications.

""

]

}

STEP 2: COMBINE PARAGRAPHS

document_names = list(documents.keys())

document_texts = []

for name in document_names:

text = " ".join(documents[name])

document_texts.append(text)

STEP 3: LOAD SENTENCE EMBEDDING MODEL

print("Loading embedding model...")

embedding_model = SentenceTransformer(

"all-MiniLM-L6-v2"

)

STEP 4: CREATE TRUE EMBEDDINGS

document_embeddings = embedding_model.encode(

document_texts,

convert_to_numpy=True

)

```
# STEP 5: LOAD EXTRACTIVE QA MODEL
print("Loading Question Answering model...")

qa_tokenizer = AutoTokenizer.from_pretrained(
    "distilbert-base-cased-distilled-squad"
)

qa_model = AutoModelForQuestionAnswering.from_pretrained(
    "distilbert-base-cased-distilled-squad"
)

# STEP 6: TAKE QUESTION FROM USER
question = input("\nEnter your Computer Networks question: ")

# STEP 7: CREATE QUESTION EMBEDDING
question_embedding = embedding_model.encode(
    [question],
    convert_to_numpy=True
)

# STEP 8: CALCULATE COSINE SIMILARITY
similarities = cosine_similarity(
    question_embedding,
    document_embeddings
)[0]

# STEP 9: FIND MOST RELEVANT DOCUMENT
best_index = similarities.argmax()

best_document = document_names[best_index]

best_score = similarities[best_index]

best_context = document_texts[best_index]

# STEP 10: DISPLAY SEMANTIC SEARCH RESULT
```

```
print("\nSEMANTIC SEARCH RESULT\n")

print("Question:", question)

print("Most Relevant Document:", best_document)

print("Cosine Similarity:", round(float(best_score), 4))


# STEP 11: DISPLAY ALL SIMILARITY SCORES
print("\nDocument Similarity Scores:")

for i in range(len(document_names)):
    print(
        document_names[i],
        "->",
        round(float(similarities[i]), 4)
    )


# STEP 12: TOKENIZE QUESTION AND CONTEXT
inputs = qa_tokenizer(
    question,
    best_context,
    return_tensors="pt",
    truncation=True
)


# STEP 13: RUN EXTRACTIVE QA MODEL
with torch.no_grad():
    outputs = qa_model(**inputs)


# STEP 14: FIND ANSWER START AND END POSITIONS
start_scores = outputs.start_logits
end_scores = outputs.end_logits

start_index = torch.argmax(start_scores, dim=1).item()
end_index = torch.argmax(end_scores, dim=1).item()
```

```
# Make sure the end position is not before the start position
```

```
if end_index < start_index:
```

```
    end_index = start_index
```

```
# STEP 15: EXTRACT ANSWER
```

```
answer_tokens = inputs["input_ids"][0][  
    start_index:end_index + 1  
]
```

```
answer = qa_tokenizer.decode(  
    answer_tokens,  
    skip_special_tokens=True  
)
```

```
# STEP 16: CALCULATE CONFIDENCE
```

```
start_probability = torch.softmax(  
    start_scores, dim=1  
)[0][start_index]
```

```
end_probability = torch.softmax(  
    end_scores, dim=1  
)[0][end_index]
```

```
confidence = start_probability * end_probability
```

```
# STEP 17: DISPLAY EXTRACTIVE QA RESULT
```

```
print("\nEXTRACTIVE QUESTION ANSWERING\n")
```

```
print("Answer:", answer)
```

```
print("Confidence:", round(float(confidence), 4))
```

SEMANTIC SEARCH AND EXTRACTIVE QUESTION ANSWERING

Loading embedding model...

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Loading weights: 100% 103/103 [00:00<00:00, 3674.24it/s]

Loading Question Answering model...

Loading weights: 100% 102/102 [00:00<00:00, 4245.71it/s]

Enter your Computer Networks question: what is TCP?

SEMANTIC SEARCH RESULT

Question: what is TCP?

Most Relevant Document: Transport Layer TCP and UDP

Cosine Similarity: 0.6714

Document Similarity Scores:

OSI and TCP/IP Models -> 0.4856

Physical and Data Link Layer -> 0.3036

Network Layer and Routing -> 0.3336

Transport Layer TCP and UDP -> 0.6714

Application Layer HTTP DNS and FTP -> 0.456

EXTRACTIVE QUESTION ANSWERING

Answer: connection - oriented

Confidence: 0.9876

RESULT

The Semantic Search + Extractive Question Answering system was successfully implemented.

The system was able to:

1. Store a collection of documents.
2. Generate true sentence embeddings for the documents.
3. Generate an embedding for the user's question.
4. Calculate cosine similarity between the question and documents.
5. Retrieve the most relevant document.
6. Use the retrieved document as context for Extractive Question Answering.
7. Extract the relevant answer from the selected document.

Thus, semantic search and extractive Question Answering were successfully combined into a single NLP application.

LEARNING OUTCOMES

1. Learned the concept of semantic search.
2. Learned how sentence embeddings represent the semantic meaning of text.
3. Learned how to generate embeddings using Sentence Transformers.
4. Learned how to calculate cosine similarity between embeddings.
5. Learned how to retrieve the most relevant document from a document collection.
6. Learned the concept of Extractive Question Answering.
7. Learned how an extractive QA model extracts an answer from a given context.
8. Learned how semantic search and Question Answering can be combined into a single NLP pipeline.
9. Understood the difference between document retrieval and answer extraction.
10. Gained practical experience with embedding-based NLP applications and Transformer-based Question Answering.